

SEMINAR • KNOWLEDGE FOR LLMs

LLM에게 우리의 지식을 가르치는 법

위키로 담고, RAG로 찾고,
그래프로 잇고, 온톨로지로 다듬기

지식 정리의 이론부터, 에이전트 프롬프트로 하는 실습까지 — 90분.

오늘 우리가 도달할 목표

복사

이 문서 폴더를 읽고,
신입도 이해할 수 있게 답해줘.
근거가 없으면 없다고 말해줘.

← 이 요청이 **잘 동작하는 지식**을 만드는 게 오늘의 전부입니다.

아주 똑똑한 신입사원이 왔습니다 — 단, 우리 회사는 처음

이미 잘 아는 것

파이썬 문법, 마케팅 이론, 통계, 영어 작문, 세상의 상식 — 인터넷에 공개된 일반 지식은 웬만한 전문가 수준.

전혀 모르는 것

우리 팀 프로젝트 이름, 사내 용어와 약어, 지난달 회의에서 결정된 것, 누가 무엇을 담당하는지 — 회사 안의 지식은 0.

문제는 지능이 아니라 **입사 교육**입니다.

오늘 배우는 것 = LLM을 위한 온보딩 자료를 만드는 기술.

오늘의 지도

키워드 4개 — 답고, 찾고, 잇고, 다듬기

LLM 위키 CH 2

지식을 담은 그릇. 사람과 AI가 함께 읽는 문서를 어떻게 쓰는가.

지식 그래프 CH 4

지식을 점과 선으로 잇기. "누가 무엇과 연결되나"에 답하는 법.

RAG CH 3

필요한 만큼만 찾아서 먹이기. 임베딩·벡터 DB·청킹이 여기 등장.

온톨로지 CH 5

용어와 규칙의 설계도. 같은 것을 같은 이름으로 부르게 하기.

순서가 곧 성숙도입니다 — 앞이 부실하면 뒤가 소용없습니다. 후반부엔 **실습 7개**로 전부 직접 만들어 봅니다.

LLM의 지식은 두 곳에 삽니다

파라미터 지식 (뇌에 새겨진 것)

학습 때 수조 개 단어에서 흡수한 지식. 고정되어 있고, 학습 마감일 (knowledge cutoff) 이후는 모릅니다.

고치려면? → 재학습·파인튜닝. 느리고 비쌉니다.

컨텍스트 지식 (손에 쥐여준 것)

대화창(컨텍스트)에 넣어준 문서·데이터. 즉시 반영되고, 출처가 분명하고, 우리가 통제합니다.

약점은 하나 — 창의 크기에 한계가 있다는 것.

오늘 배우는 모든 기술 = "적절한 지식을, 적절한 순간에, 컨텍스트로 넣는 기술"

컨텍스트 창 자체의 관리법은 별도 세미나 [「컨텍스트 엔지니어링」](#)에서 다뤘습니다.

모르면 "모른다"가 아니라 **지어냅니다**

근거 문서 없이 물어보면...

나: P-100 프로젝트 담당자가 누구야?

AI: P-100 프로젝트는 김민수 책임이
리드하고 있으며, 2025년 3분기에
시작된 것으로 알려져 있습니다.
→ 전부 창작. 그런데 말투는 확신에 참.

왜? LLM은 "가장 그럴듯한 다음 말"을 만드는 기계입니다. 근거가 없
을수록 **더 유창하게** 창작합니다.

이것이 **환각(hallucination)** — 지식 문제이지, 지능 문제가 아닙니다.



환각의 처방 1순위는 모델 교체가 아니라 근거 제공 — "이 문서 안에서만 답해. 없으면 없다고 해."

지식을 먹이는 세 가지 방법

방법	하는 일	강점	한계
① 통째로 넣기	문서 전체를 컨텍스트에 첨부	가장 단순·정확. 문서가 적으면 이걸로 끝	컨텍스트 한계, 매번 비용
② RAG 오늘의 주인공	질문과 관련된 조각만 검색해 주입	문서가 아무리 많아도 OK, 항상 최신	검색이 틀리면 답도 틀림
③ 파인튜닝	모델 가중치 자체를 재학습	말투·형식·도메인 문체 학습	사실 지식 주입엔 비효율, 갱신마다 재학습

실무 감각: ①부터 시도하세요. 문서 수십 개까지는 ①이最強입니다. 넘치기 시작할 때 ②로 — ③은 마지막 카드.

정리 안 된 지식은 넣어도 소화불량

LLM이 잘 소화하는 지식의 4가지 조건 — 그대로 다음 챕터(위키)의 설계 원칙이 됩니다.

텍스트로 존재할 것

스캔 이미지, 녹음, 머릿속 ❌ → 텍스트로 변환. 변환도 에이전트에게: "이 PDF를 마크다운으로"

자기완결적일 것

"그 건은 저번에 말한 대로" ❌ → 이름·날짜·결정 내용을 그 자리에. 한 조각만 읽어도 말이 되게.

구조가 있을 것

제목·목록·표가 있는 문서는 기계가 지도를 그릴 수 있는 문서입니다.

최신 + 출처가 있을 것

작성일·담당자를 명시하고, 낡은 문서엔 폐기 표시. AI는 3년 전 규정과 어제 규정을 구분 못 합니다 — 우리가 표시해야.

우리 팀 지식은 지금 어디에 있나요?

💬 메신저 스레드 어딘가

검색하면 나오긴 함... 아마도

✉ 메일함 "참조: 전체" 속

첨부파일 v3_final_진짜최종.pptx

📊 발표자료 23번 슬라이드

그 표가 어느 파일이더라

💻 각자의 로컬 폴더

공유된 적 없음

🧠 퇴사 예정인 김 책임 머릿속

D-30

에이전트가 읽을 수 없는 곳의 지식은, LLM에게는 **없는** 지식입니다.

해법: 파일로, 한곳에, 읽을 수 있는 형식으로 — 그게 위키입니다.

독자가 하나 늘었다 — 이제 AI도 읽습니다

	사람만 읽던 시절	AI도 읽는 시절
형식	예쁜 서식, 스크린샷 위주	텍스트와 구조 위주 (이미지엔 설명 텍스트)
발견	"어디 있는지 아는 사람"에게 물어봄	검색으로 발견되도록 제목·용어를 명확히
단위	긴 종합 문서 하나	자기완결 문서 여러 개 + 링크
용어	팀만 아는 약어 남발	약어 첫 등장 시 풀어서 정의

좋은 소식: 사람에게 좋은 문서는 대체로 LLM에게도 좋습니다. 기준이 조금 더 엄격해질 뿐입니다 — AI는 눈치가 없으니까요.

공용어는 Markdown입니다

아폴로 프로젝트

> 상태: 진행중 · 담당: 김민수 · 2026-07-01

개요

스마트팩토리 TF의 예지보전 시스템.

결정사항

- 납품처: [ACME](./acme.md)
- 1차 오픈: 9월 15일

✓ 구조가 텍스트 그 자체 — 제목·표·링크가 토큰 몇 개로 표현 (워드·PPT 대비 압도적 효율)

✓ 모든 에이전트의 모국어 — LLM은 마크다운으로 학습되고 마크다운으로 말합니다

✓ git과 찰떡 — 지식의 변경 이력·리뷰·되돌리기가 공짜

✓ 어디서나 렌더링 — GitHub, 오피시디언, 사내 위키...

💬 기존 문서 변환도 에이전트 일

이 폴더의 워드/PPT/PDF 문서를 전부 마크다운으로 변환해줘. 표는 마크다운 표로, 이미지는 내용을 설명하는 텍스트로.

LLM 친화적 문서의 4원칙

1 제목 계층 = 지도

→ ## → ### 를 성실하게. 나중에 문서를 자를 때(청킹) 이 경계가 칼자국이 됩니다.

2 한 문서 = 한 주제

"이것저것 정리.md" ❌ — 주제가 섞이면 검색도, 갱신도, 인용도 애매해집니다.

3 섹션은 자기완결

"위에서 말한 그 방식" ❌ → 고유명사로. 어느 섹션만 오려 읽어도 성립해야 합니다.

4 머리에 메타데이터

작성일 · 담당 · 상태를 문서 맨 위에 —

> 상태: 확정 · 담당: 김민수 · 2026-07-01

이 원칙들, 어디서 봤죠? — 앞 장 "좋은 먹이의 조건" 4가지가 문서 작법으로 번역된 것입니다.

에이전트를 위한 정문을 만드세요

CLAUDE.md

프로젝트 폴더의 안내판. Claude Code가 자동으로 읽고 시작합니다 — 폴더 소개, 문서 규칙, 답변 규칙을 적는 곳.

llms.txt

웹사이트가 LLM에게 주는 한 장짜리 지도.
/llms.txt 경로에 두는 공개 관례입니다.

INDEX.md

사람+AI 공용 목차. 전체 문서의 제목과 한 줄 요약 — 에이전트가 뭘 읽을지 고르는 출발점.

CLAUDE.md 예시 (축약)

이 폴더는 스마트팩토리 TF의 팀 위키다. 질문을 받으면 INDEX.md부터 본다.

용어는 GLOSSARY.md 기준을 따른다. 문서에 근거가 없으면 "문서에 없음"이라고 답한다.

문서 1,000개가 되는 날 — 위키의 벽

잘 정리된 위키도 커지면 통째로는 못 먹입니다.

컨텍스트 창 (~20만 토큰)

문서 수십 개 분량 — 여기까진 통째로 OK

팀 위키 전체 (문서 1,000개 ≈ 수백만 토큰)

들어가지 않는 나머지 93%

그래서 전략이 바뀝니다 — "전부 넣자"에서 "질문과 관련된 조각만 골라 넣자"로.

그 "골라내기"의 기술이 바로 다음 챕터, RAG입니다.

RAG — 오픈북 시험을 보게 하라

Retrieval-Augmented Generation, 검색 증강 생성 — 외우게 하지 말고, 찾아보며 답하게.

1

 Retrieval · 검색

질문과 관련된 문서 조각을 지식 창고에서 찾는다



2

 Augmented · 증강

찾은 조각을 프롬프트에 근거로 끼워 넣는다



3

 Generation · 생성

근거를 보면서 답한다 — 출처 표시 가능, 환각 급감

사실 여러분은 이미 쓰고 있습니다 — 에이전트가 파일을 찾아 읽고 나서 답할 때, 그게 RAG의 동작 원리입니다.

RAG의 성패는 1단계 검색에서 갈립니다

엉뚱한 조각을 주면

모델은 그걸 근거 삼아 자신 있게 잘못 답합니다. 생성이 아무리 좋아도 검색이 틀리면 끝 — Garbage in, garbage out.

그런데 "관련된 조각"을 어떻게 찾죠?

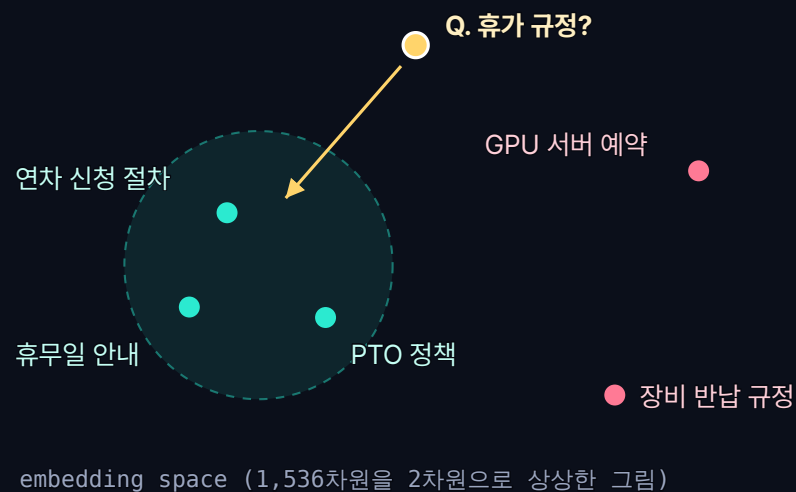
질문: "휴가 규정 알려줘"

문서에는... "연차 신청 절차", "휴무일 안내", "PTO 정책"

겹치는 단어가 하나도 없습니다.

단어가 아니라 **의미**로 찾아야 합니다 — 그 열쇠가 다음 장의 임베딩.

임베딩 — 의미를 좌표로 바꾸다



임베딩 모델이 텍스트를 숫자 목록(벡터)으로 변환합니다 — 예: 1,536개의 좌표값.

의미가 비슷하면 좌표가 가깝습니다.

"휴가 규정"과 "연차 신청"은 단어가 달라도 이웃.

검색 = "질문의 좌표에서 가장 가까운 문서 조각 찾기"라는 기하학 문제가 됩니다.

벡터 DB — 임베딩의 참고

 저장할 때 (인덱싱 — 미리 한 번)

문서
위키의 .md 파일들



청킹
조각으로 자르기



임베딩
조각마다 좌표 계산



벡터 DB
좌표+원문+출처 저장

 질문할 때 (검색 — 매번)

질문
"휴가 규정?"



임베딩
질문도 좌표로



Top-K 검색
가장 가까운 조각 K개 (유사도 점수순)



LLM
조각을 근거로 답변

도구 이름들 —

Chroma

FAISS

pgvector

Pinecone

지금은 이름만 알면 충분합니다. 코사인 유사도 = "두 좌표의 방향이 얼마나 비슷한가"를 점수 하나로 만든 것.

청킹 — 자르는 법이 답변 품질을 정한다

✗ 너무 크게

조각 하나에 주제 셋 — 관련 없는 내용까지 딸려 들어가 컨텍스트 낭비, 초점 흐림.

✗ 너무 잘게

"그 정책은 즉시 시행한다."
...무슨 정책이? 누가? — 문맥이 잘려나가 조각만으론 무의미.

✓ 구조 따라

제목(##) 경계로 자르기 — 사람이 설계한 의미 단위가 그대로 조각이 됩니다.

CH 2의 원칙이 여기서 회수됩니다 — 제목 계층이 성실하고 섹션이 자기완결적인 문서는, 청킹이 **공짜로** 잘 됩니다.

실전 팁: 조각마다 출처(파일명·섹션 경로·날짜)를 함께 저장 — 인용과 최신성 판단의 재료.

키워드 vs 벡터 vs 하이브리드

방식	원리	강한 곳	약한 곳
키워드 grep · BM25	단어가 실제로 등장하는 곳	고유명사 · 코드 · 오류메시지 · 정확한 식별자	동의어, 말바꿈
벡터 임베딩	의미가 가까운 곳	동의어 · 개념 질문 · 다국어	"P-100" 같은 정확한 식별자
하이브리드	둘 다 돌리고 합산 (+재순위화)	실무 기본값 — 서로의 구멍을 메움	

Claude Code가 코드베이스에서 `grep`을 즐겨 쓰는 이유 — 함수명·에러문구·사번·프로젝트 코드는 키워드가 이깁니다. 벡터가 만능이 아닙니다.

RAG가 넘어지는 4가지 자리

1 검색 미스

정답 조각이 Top-K에 못 들.

→ 처방: 하이브리드 검색, 질문 재작성, K 늘리기

2 문맥 잃은 조각

잘린 조각에 "누가·언제·어느 프로젝트"가 없음.

→ 처방: 자기완결 섹션 + 조각에 메타데이터 부착

3 낡은 문서가 이김

3년 전 규정이 검색 1위로.

→ 처방: 날짜 명시, 폐기 문서 표시(deprecated), 최신순 가중치

4 "전체를 종합해줘"

"우리 팀 프로젝트 전체의 리스크를 요약해줘" — 조각 5개로는 전체를 못 봅니다.

→ 처방: 다음 챕터에서 🙌

1~3번의 공통 처방은 결국 CH 2로 돌아가라 — RAG 튜닝보다 문서 정리가 먼저인 경우가 많습니다.

요즘 방식 — 에이전트가 스스로 검색합니다

고전 RAG 파이프라인

질문 → 검색 1회 → 답변.

고정된 배관 — 검색이 빗나가면 그대로 오답.

Agentic RAG

에이전트가 질문을 쪼개고, 검색하고, 부족하면 다시 검색하고, 원문을 읽어 교차 확인한 뒤 답변.

에이전트의 실제 행동 로그 (축약)

`grep` "휴가" → 0건. 검색어를 바꿔본다

`grep` "연차" → 3건. `policy/leave.md` 발견 → 전체 읽기

문서 상단: "2024-01 개정판이 최신" → 개정판을 다시 찾아 읽음

→ 근거 2건 인용해 답변

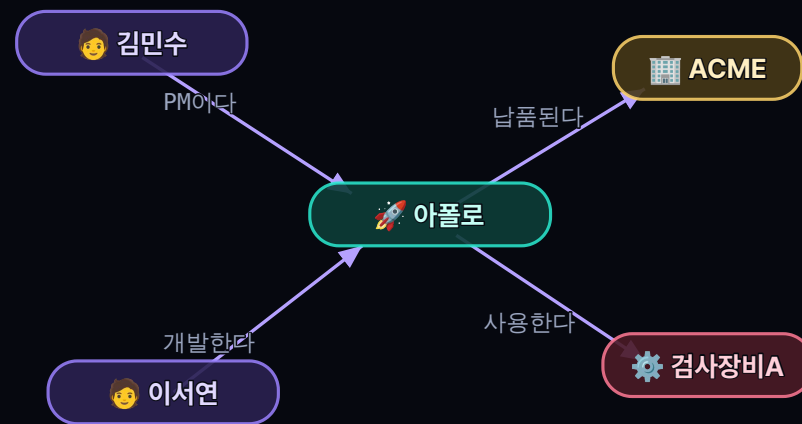
검색이 "파이프라인"에서 "행동"이 됐습니다. 그래도 기본기(잘 쓰인 문서 · 청킹 · 인덱스)의 가치는 그대로 — 에이전트도 정리된 창고에서 더 빨리, 더 정확히 찾습니다.

"누가 무엇과 연결되나"는 검색으로 안 풀립니다

"ACME에 영향을 주는 진행 중 프로젝트와 담당자를 전부 알려줘"

RAG의 대답: 가장 유사한 조각 5개... 어딘가의 문서 조각에는 전체 그림이 없습니다. 연결 관계는 여러 문서에 흩어져 있으니깐요.

지식 그래프 = 개체(점)와 관계(선)로 지식을 표현한 것. 흩어진 사실이 걸어갈 수 있는 길이 됩니다.

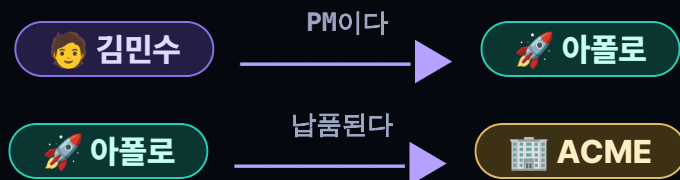


지식의 원자 — 주어 · 관계 · 목적어

그래프는 트리플(triple)이라는 3요소 문장의 모음입니다.

회의록의 한 문장

"아폴로 프로젝트는 김민수 책임이 PM을 맡고, ACME에 납품된다."



문장 속에 갇혀 있던 사실이, 기계가 따라 걸을 수 있는 길이 됩니다 — 그리고 이 변환(문장 → 트리플)을 **LLM이** 아주 잘합니다.

그래프의 주특기 — 다리 건너기

연결 찾기

"이서연과 ACME의 접점은?"

이서연 →(개발)→ 아폴로 →(납품)→
ACME

두 다리 건너면 답 — multi-hop 질문.

영향 분석

"검사장비A를 교체하면 뭐가 영향받지?"

장비 노드에서 선을 거꾸로 따라가면 프로젝트·고객·담당자가 줄줄이.

전체 조망

"진행 중 프로젝트 전부와 담당자 목록"

조각 검색이 아니라 그래프 순회 — 빠짐없이 나옵니다.

한 줄 비교 — RAG는 조각을 읽고, 그래프는 **관계를 걷습니다**. 앞 장 실패 모드 4번("전체 종합")의 해법이 여기 있습니다.

GraphRAG — 검색과 그래프의 합체

1 문서
팀 위키 (원천은 언제나 문서)

→

2 LLM 추출
개체·관계(트리플)를 자동 추출

→

3 그래프 구축
+ 군집별 요약문 미리 생성

→

4 질문 시
그래프 탐색 + 조각 검색을 함께 사용

얻는 것

"전체 종합" 질문이 풀림 — 세부 질문은 조각 검색으로, 조망 질문은 그래프·군집 요약으로.

치르는 것

구축 시 문서 전체에 LLM 호출 — 비용과 시간. 문서가 바뀌면 그래프도 갱신해야 합니다.

Microsoft가 2024년 공개한 GraphRAG가 이 조합을 대중화했습니다 — 이후 변형이 계속 등장 중.

전부 그래프로 만들려다 아무것도 못 만듭니다

1 개체 종류는 5~10개만

사람 · 프로젝트 · 고객사 · 시스템/장비 · 문서 — 이 정도로 시작. 세상 만물을 노드로 만들지 마세요.

2 관계도 5개 이내로

담당한다 · 납품한다 · 사용한다 · 의존한다 · 결정했다 — 관계가 많을수록 추출 품질이 떨어집니다.

3 추출은 LLM, 검수는 사람

에이전트가 뽑고, 사람이 훑어보며 오추출만 걷어내기 — 실습 4에서 해봅니다.



정직한 조언: 문서 100개 미만이면 위키 + 에이전트 검색으로 충분한 경우가 많습니다. 그래프는 "관계 질문이 실제로 자주 나올 때" 도입하세요.

같은 것을 세 가지 이름으로 부르는 회사

같은 회사, 같은 대상, 문서 셋
회의록: "고객사 ACME 요청사항..."
계약서: "거래처 ACME와의 계약..."
주간보고: "클라이언트 측 피드백..."

상태 표기도 제각각
"완료" vs "종결" vs "Done" vs "✅"

사람은 눈치로 통일해서 읽습니다.

LLM은 셋을 다른 개념으로 취급할 수 있습니다.

그래프를 만들면? — "고객사 ACME"와 "거래처 ACME"가 노드 2개로 쪼개져 연결이 끊깁니다.

온톨로지 = 우리 세계에 어떤 개념이 존재하고, 뭐라고 부르고, 어떤 관계가 허용되는지에 대한 약속 — 지식의 설계도.

폴더에서 온톨로지까지 — 구조화의 사다리

아래에서 위로 갈수록 기계가 할 수 있는 일이 늘어납니다. (전부 오를 필요는 없습니다)

4 · 온톨로지

분류 + 관계 + 제약 — "프로젝트는 반드시 PM 1명을 갖는다", "장비는 프로젝트에 사용된다". 규칙 검사와 추론이 가능해짐.

3 · 택소노미

위계가 있는 분류 체계 — 장비 > 검사장비 > 비전검사기. "상위 개념"이 생김.

2 · 태그

한 문서에 라벨 여러 개 — #아폴로 #보안 #결정사항. 서랍 여러 개에 동시에.

1 · 폴더/네이밍

물건을 서랍에 — projects/apollo/회의록_0701.md. 위치가 곧 의미.

대부분의 팀은 1~2단에 있습니다. 오늘 목표는 4단의 미니 버전을 맛보는 것.

설계도의 부품 5가지

부품	뜻	예
클래스	개념의 종류	사람, 프로젝트, 고객사
인스턴스	실제 개체	김민수, 아폴로, ACME
속성	개체가 갖는 값	사번, 마감일, 상태
관계	개체 사이의 연결	담당한다, 납품한다
제약	지켜야 할 규칙	PM은 정확히 1명

```
# ontology.yaml - 미니 온톨로지 (발췌)
classes:
  프로젝트:
    속성: [이름, 상태, 마감일]
    제약:
      - 담당 PM은 정확히 1명
      - 상태는 [준비, 진행중, 완료] 중 하나
relations:
  - 사람 --(담당한다)--> 프로젝트
  - 프로젝트 --(납품된다)--> 고객사
```

형식은 자유 — 핵심은 기계도 사람도 읽을 수 있는 한 장의 약속.

표준의 세계 — 이름만 알아두세요

이름	정체	한 줄 요약
RDF	트리플의 표준 포맷	(주어, 관계, 목적어)를 기계끼리 교환하는 규격
OWL	온톨로지 기술 언어	클래스·관계·제약을 엄밀하게 적는 언어 (추론기가 검증)
SPARQL	그래프 질의어	그래프계의 SQL — "아폴로의 PM은?"을 쿼리로
schema.org	웹의 공용 온톨로지	구글 검색의 리치 결과(평점·레시피·일정)가 이걸로 동작

언제 표준이 필요한가

학계 · 공공데이터 · 조직 간 교환 · 규제 대응 — 바깥과 연결될 때.

사내에서는

마크다운 + YAML로 시작하면 충분합니다. 필요해지면 그때 표준으로 승격 — 개념은 이미 같습니다.

온톨로지는 LLM의 스타일 가이드입니다

추출의 기준

문서에서 트리플을 뽑을 때 — *"이 클래스와 관계만 사용해"*. 들쭉날쭉하던 추출이 일관해집니다.

검증의 기준

제약 위반을 자동 검출 — *"PM 없는 프로젝트를 찾아줘", "상태 값이 규정 밖인 문서는?"*

대화의 기준

용어사전을 컨텍스트에 넣으면 답변 용어가 통일 — "거래처"라고 물어도 "고객사" 기준으로 답합니다.

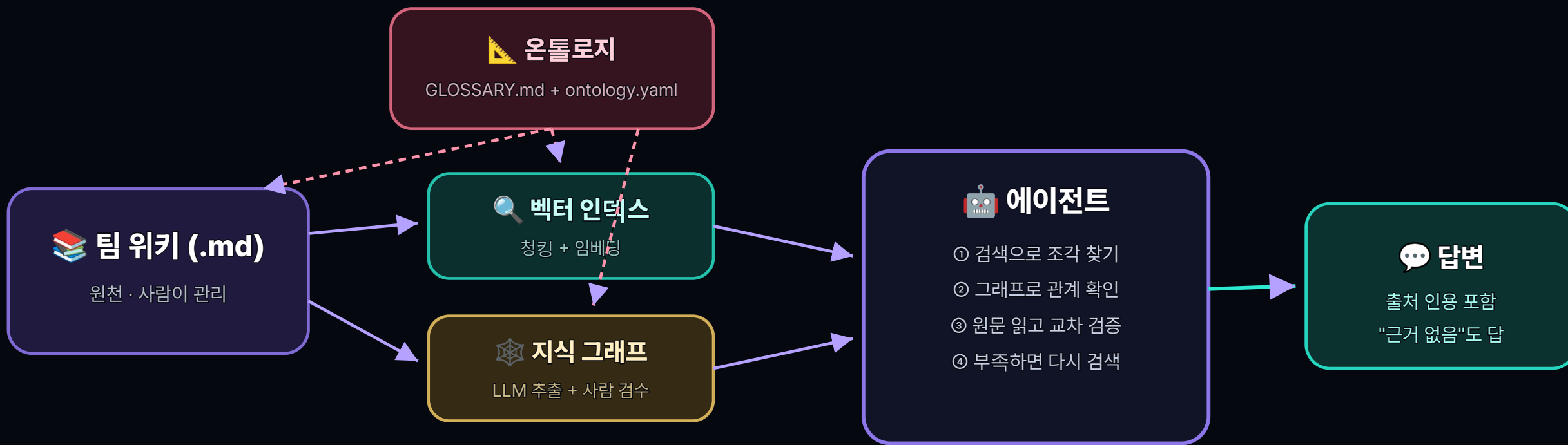
시작은 거창한 OWL이 아니라 — *GLOSSARY.md(용어사전) + ontology.yaml(스키마) 두 파일이면 됩니다. LLM 시대에 온톨로지는 "만들기 어려운 것"에서 "에이전트와 함께 반나절이면 초안 나오는 것"이 되었습니다.*

네 가지 도구, 한 장 정리

도구	은유	잘 받는 질문	시작 비용
 LLM 위키	지식의 그릇	"~가 뭐야? 어떻게 해?" (내용 질문 전부의 원천)	낮음 오늘 시작 가능
 RAG	사서(司書)	"~어디 있었지? 뭐라고 돼 있지?"	낮음 에이전트면 즉시
 지식 그래프	지도	"~와 ~의 관계는? 전체 목록은? 영향 범위는?"	중간 추출+검수 필요
 온톨로지	문법책	(질문보다) 위 셋의 품질을 끌어올림	중간 두 파일부터

넷은 경쟁자가 아니라 **층**입니다 — 1층(위키)이 부실하면 위층 전부가 흔들립니다. 투자 순서도 이 순서.

팀 지식봇의 설계도



- - - 온톨로지는 나머지 전부의 "기준"으로 작동

지금부터의 실습에서, 이 그림의 **미니어처를 전부 직접** 만듭니다.

👉 실습 0 · 실습장 만들기 (5분)

가상의 팀 문서 세트를 생성합니다

준비물: Claude Code(또는 다른 에이전트 도구) + 빈 폴더. 여기서 만든 폴더를 실습 6까지 계속 씁니다.

💬 에이전트에게 이렇게 말하세요

복사

knowledge-lab 폴더를 만들어줘. 가상의 "스마트팩토리 TF" 팀이 있다고 치고, 실습용 문서 6개를 만들어줘: 회의록 2개(결정사항 포함), 프로젝트 소개 1개, 장비 매뉴얼 1개, 신입 온보딩 메모 1개, 2년 전의 낡은 규정 1개. 일부러 현실적인 문제를 심어줘 — 같은 고객사를 "고객사/거래처/클라이언트"로 섞어 부르고, 담당자 정보를 여러 문서에 흩어 놓고, 대명사("그 건")도 좀 넣어줘.

✅ 확인 포인트

문서 6개가 생겼는지, 용어 불일치·흩어진 담당자 정보가 정말 들어갔는지 훑어보기.
→ 이 "엉망인 상태"가 우리의 출발점입니다. 진짜 팀 문서로 해도 좋아요(보안 등급만 확인!).

👉 실습 1 · 위키화 (10분)

흩어진 메모를 LLM 위키로

CH 2의 4원칙을 그대로 에이전트에게 규칙으로 줍니다.

💬 에이전트에게 이렇게 말하세요

복사

knowledge-lab의 문서들을 LLM 친화적 위키로 재구성해서 wiki/ 폴더에 만들어줘.

규칙: ① 문서 맨 위에 제목·날짜·담당·상태 메타데이터

② 한 문서 한 주제 (필요하면 쪼개기) ③ 대명사 "그 건" 대신 고유명사

④ 제목 계층(#, ##)을 성실하게 ⑤ wiki/INDEX.md에 전체 목차 + 문서별 한 줄 요약.

낡은 규정에는 최신 문서와 모순되는 부분에 "폐기됨" 표시를 달아줘. 원본은 archive/로.

✅ 확인 포인트

INDEX.md만 읽고 전체 구조가 보이는가? · 아무 섹션이나 하나 잘라 읽어도 말이 되는가?
(= 자기완결 테스트, 나중에 청킹 품질이 된다)

👉 실습 2 · 진입점 (5분)

CLAUDE.md — 에이전트의 정문 달기

💬 에이전트에게 이렇게 말하세요

복사

knowledge-lab에 CLAUDE.md를 만들어줘. 내용: ① 이 폴더가 무엇인지 두 줄 소개
② 질문을 받으면 wiki/INDEX.md부터 볼 것 ③ 문서 규칙(메타데이터·용어) 요약
④ 답변 규칙: 반드시 출처 파일을 인용하고, 문서에 근거가 없으면
"문서에 없음"이라고 답할 것 — 추측 금지.

✎ 테스트 A — 있는 지식

복사

(새 세션에서) 아폴로 프로젝트 PM이 누구야?

✎ 테스트 B — 없는 지식

복사

(새 세션에서) 우리 팀 보너스 지급 규정 알려줘.

A는 출처와 함께 답해야 하고, B는 "문서에 없음"이 나와야 성공 — 환각 방지 장치가 작동하는 순간입니다.

청킹 → 검색 → 인용 답변을 눈으로 봅니다

💬 에이전트에게 이렇게 말하세요

복사

wiki/ 문서를 ## 제목 경계로 청킹해서 chunks.json으로 저장해줘.
각 청크에 출처(파일명·섹션 경로·날짜)를 붙여줘.
그다음 질문 "고객사 납품 일정 알려줘"에 대해 두 방식으로 검색해봐:
① 키워드 매칭으로 Top-3 청크 ② 의미 관련도로(네가 직접 판단) Top-3 청크.
두 결과를 표로 비교해서 보여주고, 마지막으로 ②의 청크만 근거로
출처 인용을 달아 답변해줘.

✅ 확인 포인트

- ①과 ②의 Top-3가 다른가? (같은 질문, 다른 검색 = CH3 19장 체험)
- "납품"이라는 단어가 없는 문서를 ②가 찾아냈는가? → 그게 의미 검색의 가치
- 이번엔 "거래처 일정"으로 물어보기 - 용어 불일치가 검색을 흐드는 걸 관찰 (실습 5 예고)

임베딩 API 없이 에이전트의 판단으로 의미 검색을 흉내 낸 것 — 개념 체험이 목적입니다. 실제 시스템은 임베딩+벡터 DB로 같은 일을 대규모로 합니다.

문서에서 그래프를 뽑아 그립니다

💬 에이전트에게 이렇게 말하세요

복사

wiki/ 전체에서 (주어, 관계, 목적어) 트리플을 추출해 triples.csv로 저장해줘.
개체 종류는 사람/프로젝트/고객사/장비 4가지만,
관계는 담당한다/납품한다/사용한다/결정했다 4가지만 허용해.
각 트리플에 근거 문서도 적어줘.
그다음 이 그래프를 mermaid 다이어그램으로 그린 graph.html을 만들어서 열어줘.
개체 종류별로 색을 다르게.

✎ 그래프로 질문하기 (multi-hop)

복사

triples.csv만 보고 답해줘:
ACME와 연결된 사람을 전부, 연결 경로와 함께.

✅ 확인 포인트

"고객사 ACME"와 "거래처 ACME"가
노드 2개로 쪼개졌는가? → 온톨로지가
필요한 순간. 다음 실습에서 고칩니다.

용어사전 + 스키마 = 미니 온톨로지

💬 에이전트에게 이렇게 말하세요

복사

- ① wiki/에서 같은 대상을 다르게 부르는 용어들을 찾아 GLOSSARY.md를 만들어줘 — 대표어·동의어 목록·한 줄 정의.
- ② ontology.yaml을 만들어줘 — 개체 클래스 4종의 속성, 허용 관계 4종, 그리고 제약(예: 프로젝트에는 담당 PM이 정확히 1명).
- ③ 이 두 파일을 기준으로 실습 4의 트리플을 다시 추출해서, 그래프가 어떻게 달라졌는지 전/후를 비교해줘.
- ④ 마지막으로 제약을 위반하는 데이터가 있는지 검사해줘.

✅ 확인 포인트

- 쪼개졌던 ACME 노드가 하나로 합쳐졌는가? ← 온톨로지의 가치가 체감되는 순간
- "PM 없는 프로젝트" 같은 제약 위반이 검출됐는가? = 지식의 품질 검사가 자동화됨

CLAUDE.md에 "용어는 GLOSSARY.md 기준" 한 줄을 추가하면 — 이후 모든 대화의 용어가 통일됩니다.

👉 실습 6 · 종합 (10분)

우리 팀 지식봇 설계서 쓰기

💬 에이전트에게 이렇게 말하세요

복사

지금까지 knowledge-lab에 만든 것(위키·CLAUDE.md·청킹·그래프·온톨로지)을 바탕으로,
"우리 팀 지식봇 구축 설계서" DESIGN.md를 써줘.
먼저 우리 팀 상황(문서 수, 인원, 질문 유형, 보안 요건)을 나에게 물어보고 반영해줘.
내용: 단계별 로드맵(1단계=위키 정리부터), 각 단계에 필요한 도구와 프롬프트,
단계별 "이 신호가 오면 다음 단계로" 기준, 예상되는 함정 3가지.

진짜 팀에 적용하는 순서 — 오늘 배운 그대로:

1. 위키부터

중요 문서 10개를 마크다운으로 + INDEX + CLAUDE.md



2. 에이전트 검색

일상 질문을 에이전트에게 — 이게 이미 RAG



3. 필요할 때만

관계 질문이 찾아지면 그래프 · 용어가 꼬이면 온톨로지

이 장표만 찍어 가세요

변환  이 워드/PPT/PDF를 마크다운으로 변환해줘. 표는 표로, 이미지는 설명 텍스트로

진입점  CLAUDE.md 만들어줘 — 문서 규칙과 "근거 없으면 없다고 답하기" 포함

청킹  ## 제목 경계로 청킹해서 출처 포함 JSON으로 저장해줘

그래프 추출  문서에서 트리플을 추출해줘 — 개체 4종·관계 4종만 허용, 근거 표기

멀티홉  X와 Y의 연결 경로를 그래프에서 전부 찾아줘

스키마  ontology.yaml에 클래스·속성·관계·제약을 정의해줘

최신성  6개월 넘은 문서를 찾아 담당자별 확인 요청 목록을 만들어줘

위키화  이 폴더를 위키로 재구성해줘 — 메타데이터·한 문서 한 주제·INDEX.md 포함

근거 답변  문서에서 근거를 찾아 출처와 함께 답해줘. 없으면 없다고 해

검색 비교  이 질문을 키워드/의미 두 방식으로 검색해서 Top-3를 비교해줘

그래프 그림  트리플을 mermaid 다이어그램 HTML로 그려줘, 종류별 색 구분

용어사전  같은 대상을 다르게 부르는 용어를 찾아 GLOSSARY.md 만들어줘

품질 검사  온톨로지 제약을 위반하는 데이터를 찾아줘

배움  방금 네가 한 과정을 초보자에게 설명하듯 알려줘

같은 내용이 저장소의 knowledge/README.md에도 정리돼 있습니다 — 복사해서 쓰세요.

마무리

기억할 것은 세 문장입니다

1 LLM에게 부족한 건 지능이 아니라 **우리의 지식** — 파일로 꺼내놓는 것부터

2 거창한 시스템보다 **잘 쓰인 마크다운 10개** — 위키가 모든 것의 1층

3 검색(RAG)으로 시작하고 — **관계(그래프)와 규칙(온톨로지)** 은 필요해질 때

질문은 화면 아래 입력창으로 

함께 보면 좋은 자료 — [컨텍스트 엔지니어링](#) · [최소한의 Git & GitHub](#) · [실습 워크숍](#)